```
// 💎 Apple Deal Scanner — même moteur Vinted que le Phone Scanner du 14 octobre.
// Uniquement Apple : iPhone, iPad et MacBook, toutes générations, tous états
// (neuf, très bon état, écran / vitre / batterie à changer).
// Pas de prix fixes : le bot apprend tout seul le prix du marché de chaque modèl
// à partir des annonces Vinted qu'il lit, et n'envoie que les prix très en desso
// Chaque annonce n'est envoyée qu'UNE seule fois.
import { existsSync, readFileSync, writeFileSync } from "node:fs";

interface Listing {
  id: string;
  source: "vinted";
  title: string;
  price: number;
  currency: string;
  url: string;
  image?: string;
  extra?: string;
}

type Cat = "iphone" | "ipad" | "macbook";
interface Stream { name: string; query: string; catalog?: string }

const config = JSON.parse(readFileSync("config.json", "utf8")) as {
  everySeconds?: number;
  mainCountry?: string;
  otherCountries?: string[];
  dealRatio?: number;
  topRatio?: number;
  minSamples?: number;
  streams: Stream[];
  minPrice: Record<Cat, number>;
  unknownCap: Record<Cat, number>;
  exclude: string[];
  heavyDamage: string[];
};

const EVERY = Math.max(config.everySeconds ?? 8, 5);
const MAIN = (config.mainCountry ?? "www.vinted.fr").replace(/^https?:\/\//, "").
const OTHERS = (config.otherCountries ?? []).map((d) => d.replace(/^https?:\/\//,
const DEAL_RATIO = config.dealRatio ?? 0.5; // envoyé si prix ≤ 50 % du prix marc
const TOP_RATIO = config.topRatio ?? 0.35;
const MIN_SAMPLES = config.minSamples ?? 6;
```

```
const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));
const norm = (s: string) => s.normalize("NFD").replace(/[̀-ͯ]/g, "").toLowerCase();

// ---------- Vinted (lecture de la page de recherche, plusieurs pays possibles)
const UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
const VBASE = "https://www.vinted.fr";
const PHONES_CATALOG = "3661"; // catégorie Vinted « Téléphones portables »

// Une session (cookies + pause) par pays
interface VSession { base: string; jar: Map<string, string>; blockedUntil: number
const sessions = new Map<string, VSession>();
const sessionFor = (host: string): VSession => {
  let v = sessions.get(host);
  if (!v) { v = { base: `https://${host}`, jar: new Map(), blockedUntil: 0, strik
  return v;
};
const countryOf = (base: string) => base.split(".").pop()!.toUpperCase();

function storeCookies(v: VSession, res: Response) {
  for (const c of res.headers.getSetCookie()) {
    const [pair] = c.split(";");
    const i = pair.indexOf("=");
    if (i > 0) v.jar.set(pair.slice(0, i).trim(), pair.slice(i + 1));
  }
}

const vHeaders = (v: VSession, extra: Record<string, string> = {}): Record<string
  "user-agent": UA,
  accept: "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
  "accept-language": "fr-FR,fr;q=0.9,en;q=0.7",
  "cache-control": "no-cache",
  ...(v.jar.size ? { cookie: [...v.jar].map(([k, val]) => `${k}=${val}`).join(";
  ...extra,
});

async function refreshSession(v: VSession) {
  const h = await fetch(v.base + "/", { headers: vHeaders(v), redirect: "manual"
  storeCookies(v, h);
  await h.text();
  const r = await fetch(v.base + "/web/api/auth/refresh", {
    method: "POST",
    headers: vHeaders(v, { accept: "application/json", "content-type": "applicati
    body: "{}",
    redirect: "manual",
  });
  storeCookies(v, r);
```

```
  await r.text();
  console.log(`Vinted ${countryOf(v.base)} session rafraîchie : HTTP ${r.status}`
}

async function getCatalogHtml(v: VSession, u: URL): Promise<string> {
  for (let attempt = 0; attempt < 3; attempt++) {
    const r = await fetch(u, { headers: vHeaders(v), redirect: "manual" });
    storeCookies(v, r);
    if (r.status === 403 || r.status === 429) {
      // pause progressive : 1 min, 2 min, 5 min, puis 10 min
      v.strikes++;
      const mins = [1, 2, 5, 10][Math.min(v.strikes - 1, 3)];
      v.blockedUntil = Date.now() + mins * 60_000;
      v.jar.clear();
      throw new Error(`Vinted ${countryOf(v.base)} bloque (HTTP ${r.status}) : pa
    }
    if (r.status >= 300 && r.status < 400) {
      await refreshSession(v);
      continue;
    }
    if (!r.ok) throw new Error(`Vinted HTTP ${r.status}`);
    const html = await r.text();
    if (/<title>\s*Session refresh/i.test(html)) {
      await refreshSession(v);
      continue;
    }
    v.strikes = 0;
    return html;
  }
  throw new Error("Vinted : session non valide");
}

const decode = (s: string) =>
  s.replace(/&amp;/g, "&").replace(/&quot;/g, '"').replace(/&#39;|&apos;/g, "'").

const SP = "(?:\\s|\\u00a0|\\u202f| | | )*";
// Prix « 1 250,00 € », « 250 € » ou « € 250,00 » (Pays-Bas, Irlande…) — les mill
const NUM = "(\\d{1,3}(?:[ .,\\u00a0\\u202f]\\d{3})+(?:[.,]\\d{1,2})?|\\d{1,5}(?:
const EUR = "(?:€|&euro;|&#8364;)";
const PRICE_AFTER = new RegExp(NUM + SP + EUR);
const PRICE_BEFORE = new RegExp(EUR + SP + NUM);
const toNumber = (raw: string) => {
  let s = raw.replace(/[\s\u00a0\u202f]/g, "");
  const dec = s.match(/[.,](\d{1,2})$/);
  const int = dec ? s.slice(0, -dec[0].length) : s;
  return Number(int.replace(/[.,]/g, "") + (dec ? "." + dec[1] : ""));
```

```
};
const readPrice = (seg: string): number | null => {
  const a = seg.match(PRICE_AFTER);
  const b = seg.match(PRICE_BEFORE);
  const m = a && b ? ((a.index ?? 0) <= (b.index ?? 0) ? a : b) : a ?? b;
  return m ? toNumber(m[1]) : null;
};
const STATE_RE = /(Neuf avec étiquette|Neuf sans étiquette|Très bon état|Bon état
const IMG_RE = /https:\/\/images\d*\.vinted\.net\/[^"'\s)<>\\]+/;

function parseCatalog(html: string, base = VBASE): Listing[] {
  const out: Listing[] = [];
  const ids = new Set<string>();

  // 1) Cartes de la page (liens <a href="/items/123-titre">)
  const anchors: { id: string; slug: string; idx: number; tag: string }[] = [];
  const reA = /<a\b[^>]*?href=["'](?:https:\/\/www\.vinted\.[a-z.]+)?\/items\/(\d
  for (const m of html.matchAll(reA)) {
    if (ids.has(m[1])) continue;
    ids.add(m[1]);
    anchors.push({ id: m[1], slug: m[2], idx: m.index ?? 0, tag: m[0] });
  }
  if (anchors.length >= 3) {
    anchors.forEach((a, i) => {
      const end = Math.min(anchors[i + 1]?.idx ?? a.idx + 6000, a.idx + 6000);
      const seg = html.slice(a.idx, end);
      const price = readPrice(seg);
      if (price === null) return;
      const attr = a.tag.match(/title=["']([^"']{3,300})["']/);
      const img = seg.match(IMG_RE);
      const st = seg.match(STATE_RE);
      out.push({
        id: `vinted:${a.id}`,
        source: "vinted",
        title: attr ? decode(attr[1]) : a.slug.replace(/-/g, " "),
        price,
        currency: "EUR",
        url: `${base}/items/${a.id}-${a.slug}`,
        image: img?.[0],
        extra: st?.[1],
      });
    });
    if (out.length) return out;
  }

  // 2) Secours : données JSON de la page
```

```
  ids.clear();
  for (const m of html.matchAll(/\/items\/(\d{6,})-([a-z0-9-]*)/g)) {
    if (ids.has(m[1])) continue;
    ids.add(m[1]);
    const idx = m.index ?? 0;
    const win = html.slice(Math.max(0, idx - 1500), idx + 300).replace(/\\"/g, '"
    const all = [...win.matchAll(/"amount"\s*:\s*"?(\d+(?:\.\d+)?)/g)];
    if (!all.length) continue;
    const t = win.match(/"title"\s*:\s*"([^"]{3,200})"/g);
    const lastTitle = t ? t[t.length - 1].replace(/^"title"\s*:\s*"/, "").replace
    out.push({
      id: `vinted:${m[1]}`,
      source: "vinted",
      title: lastTitle ?? m[2].replace(/-/g, " "),
      price: Number(all[all.length - 1][1]),
      currency: "EUR",
      url: `${base}/items/${m[1]}-${m[2]}`,
      image: win.match(IMG_RE)?.[0],
    });
  }
  return out;
}

async function searchVinted(query: string, catalog: string | undefined, host: str
  const v = sessionFor(host);
  if (Date.now() < v.blockedUntil) throw new Error(`en pause (Vinted ${countryOf(
  const u = new URL(v.base + "/catalog");
  u.searchParams.set("search_text", query);
  if (catalog) u.searchParams.set("catalog[]", catalog);
  u.searchParams.set("order", "newest_first");
  u.searchParams.set("per_page", "96");
  if (page > 1) u.searchParams.set("page", String(page));
  u.searchParams.set("time", String(Math.floor(Date.now() / 1000))); // évite les
  const html = await getCatalogHtml(v, u);
  const items = parseCatalog(html, v.base);
  if (items.length === 0) {
    const k = html.indexOf("/items/");
    console.error(`Aucune annonce lisible (${countryOf(v.base)} ${query}). Extrai
  }
  return items;
}

// ---------- Reconnaissance du produit et du modèle ----------
interface Product { cat: Cat; key: string; family: string }
const clean = (s: string) => s.replace(/\s+/g, " ").trim();
```

```
function detect(title: string): Product | null {
  // MacBook : famille (Air / Pro) + puce M1…M5 ou année
  let m = title.match(/macbook\s*(pro|air)?/);
  if (m) {
    const family = clean(`macbook ${m[1] ?? ""}`);
    const chip = title.match(/\bm\s?([1-5])\s*(pro|max|ultra)?\b/);
    const year = title.match(/\b(20[012]\d)\b/);
    const gen = chip ? clean(`m${chip[1]} ${chip[2] ?? ""}`) : year ? year[1] : "
    return { cat: "macbook", family, key: clean(`${family} ${gen}`) };
  }
  // iPad : famille (Pro / Air / mini / normal) + puce, génération, taille ou ann
  m = title.match(/ipad\s*(pro|air|mini)?/);
  if (m) {
    const family = clean(`ipad ${m[1] ?? ""}`);
    const rest = title.slice((m.index ?? 0) + m[0].length);
    const chip = title.match(/\bm\s?([1-5])\b/);
    const nth = title.match(/\b(\d{1,2})\s*(?:e|eme|th|gen|generation|generacion|
    const num = rest.match(/^\s*(\d{1,2}(?:[.,]\d)?)\b/);
    const year = title.match(/\b(20[12]\d)\b/);
    const gen = chip ? `m${chip[1]}` : nth ? nth[1] : num ? num[1].replace(",", "
    return { cat: "ipad", family, key: clean(`${family} ${gen}`) };
  }
  // iPhone : toutes les générations (3G → 18, SE, X, XR, XS, Air…) + Pro / Pro M
  m = title.match(/iphone\s*(air|se|xs\s*max|xr|xs|x|1[0-8]e?|[3-8]s?|3g)?\s*(pro
  if (m) {
    if (!m[1]) return { cat: "iphone", family: "iphone", key: "iphone" };
    const gen = m[1].replace(/\s+/g, " ");
    return { cat: "iphone", family: `iphone ${gen.replace(/^(\d+)e$/, "$1")}`, ke
  }
  return null;
}

// ---------- Prix du marché (appris en continu, gardé dans market.json) --------
const MARKET_FILE = "market.json";
const market: Record<string, number[]> = existsSync(MARKET_FILE) ? JSON.parse(rea
const marketSeen = new Set<string>();
const saveMarket = () => writeFileSync(MARKET_FILE, JSON.stringify(market));

function addSample(id: string, p: Product, price: number) {
  if (marketSeen.has(id)) return;
  marketSeen.add(id);
  if (marketSeen.size > 40000) marketSeen.clear();
  for (const k of new Set([p.key, `famille:${p.family}`])) {
    const arr = (market[k] ??= []);
    arr.push(price);
    if (arr.length > 400) arr.splice(0, arr.length - 400);
```

```
  }
}

const median = (a: number[]) => {
  const s = [...a].sort((x, y) => x - y);
  const n = s.length;
  return n % 2 ? s[(n - 1) / 2] : (s[n / 2 - 1] + s[n / 2]) / 2;
};

const BARE = new Set(["iphone", "ipad", "macbook"]); // titre sans modèle précis
function marketPrice(p: Product): { value: number; samples: number } | null {
  if (BARE.has(p.key)) return null;
  const k = market[p.key];
  if (k && k.length >= MIN_SAMPLES) return { value: median(k), samples: k.length
  const f = market[`famille:${p.family}`];
  if (p.family !== p.key && !BARE.has(p.family) && f && f.length >= MIN_SAMPLES)
  return null;
}

// ---------- Filtres ----------
// Pièces et accessoires : on regarde le DÉBUT du titre (toutes langues)
const PART_START = /^(ecran|batterie|boite|boitier|vitre|dalle|lcd|oled|camera|ch
const ANDROID = /\b(samsung|galaxy|xiaomi|redmi|huawei|oppo|realme|pixel|oneplus|

// Mot trouvé seulement en début de mot : « bloqué » ≠ « débloqué », « verrouillé
const esc = (w: string) => norm(w).replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
const hasWord = (t: string, w: string) => new RegExp(`(^|[^a-z0-9])${esc(w)}`).te

const DAMAGE: [RegExp, string][] = [
  [/ecran (casse|fissure|brise|abime|raye|a changer)|vitre (avant )?(cassee|fissu
  [/dos (casse|fissure)|vitre arriere|arriere (casse|fissure)|back ?glass|rucksei
  [/batterie|autonomie|akku|bateria|batteria|accu\b/, "batterie à changer"],
  [/face ?id/, "Face ID"],
  [/camera|kamera/, "caméra"],
  [/rayure|rayee|kratzer|aranazo|graffi/, "rayures"],
];
const damageOf = (t: string) => DAMAGE.filter(([re]) => re.test(t)).map(([, label

const CAT_LABEL: Record<Cat, string> = { iphone: "📱 iPhone", ipad: "📲 iPad", ma

interface Verdict { p: Product; title: string }
// Retourne la raison du refus (avant le prix), ou le produit reconnu
function check(l: Listing): string | Verdict {
  const t = norm(l.title);
  // titre réel de l'annonce (sans « , marque : Apple, … » ajouté par Vinted, tou
  const title = t.split(/,\s*(?:marque|marke|marca|merk|brand|znacka|zimolu|preke
```

```
  const p = detect(title);
  if (!p) return "pas un produit Apple suivi";
  if (PART_START.test(title)) return "pièce ou accessoire";
  if (ANDROID.test(title)) return "pas Apple";
  const bad = config.exclude.find((w) => hasWord(title, w));
  if (bad) return `mot exclu « ${bad} »`;
  const heavy = config.heavyDamage.find((w) => hasWord(title, w));
  if (heavy) return `trop abîmé « ${heavy} »`;
  if (l.price < (config.minPrice[p.cat] ?? 20)) return `prix trop bas = accessoir
  return { p, title };
}

// ---------- Discord ----------
async function sendToDiscord(embed: Record<string, unknown>): Promise<void> {
  const url = process.env.DEALS_WEBHOOK_URL || process.env.DISCORD_WEBHOOK_URL;
  if (!url) throw new Error("DISCORD_WEBHOOK_URL manquant");
  for (let attempt = 0; attempt < 4; attempt++) {
    try {
      const r = await fetch(url, {
        method: "POST",
        headers: { "content-type": "application/json" },
        body: JSON.stringify({ username: "Apple Deal Scanner", embeds: [embed] })
      });
      if (r.ok) return;
      if (r.status === 429) {
        const j: any = await r.json().catch(() => ({}));
        await sleep(((j.retry_after ?? 2) as number) * 1000 + 300);
        continue;
      }
      if (r.status >= 500) { await sleep(2000); continue; }
      throw new Error(`Discord HTTP ${r.status}`);
    } catch (e) {
      if ((e as Error).message.startsWith("Discord HTTP")) throw e;
      await sleep(2000); // coupure réseau : on réessaie
    }
  }
  throw new Error("Discord injoignable");
}

const FLAG: Record<string, string> = {
  FR: "🇫🇷", BE: "🇧🇪", ES: "🇪🇸", IT: "🇮🇹", DE: "🇩🇪", AT: "🇦🇹", NL: "🇳🇱", LU: "🇱🇺
  IE: "🇮🇪", FI: "🇫🇮", GR: "🇬🇷", SK: "🇸🇰", LT: "🇱🇹", HR: "🇭🇷", LV: "🇱🇻", EE: "🇪🇪
};

// « pro max » → « Pro Max », « xs max » → « XS Max », « m1 pro » → « M1 Pro »
const pretty = (k: string) => k.split(" ").map((w) => (w.length <= 2 || /^m\d$/.t
```

```
function dealEmbed(l: Listing, v: Verdict, ref: ReturnType<typeof marketPrice>, h
  const ratio = ref ? l.price / ref.value : null;
  const mid = (TOP_RATIO + DEAL_RATIO) / 2;
  const tier = ratio === null ? "🆕 PETIT PRIX" : ratio <= TOP_RATIO ? "🔥🔥 AFF
  const color = ratio === null ? 0x5865f2 : ratio <= TOP_RATIO ? 0xff3b30 : ratio
  const defects = damageOf(v.title);
  const cc = countryOf(`https://${host}`);
  return {
    author: { name: tier },
    title: l.title.split(/,\s*(?:marque|marke|marca|merk|brand)\s*:/i)[0].slice(0
    url: l.url,
    color,
    fields: [
      { name: "💰 Prix", value: `**${l.price} €**`, inline: true },
      { name: "📊 Prix marché", value: ref ? `~${Math.round(ref.value)} €  ·  **-
      { name: "🏷️ Modèle", value: `${CAT_LABEL[v.p.cat]} ${pretty(v.p.key.replace
      { name: "✨ État", value: l.extra ?? "non précisé", inline: true },
      { name: "🔧 À réparer", value: defects.length ? defects.join(", ") : "rien
      { name: "🌍 Pays", value: `${FLAG[cc] ?? ""} ${cc}`.trim(), inline: true },
    ],
    ...(l.image ? { image: { url: l.image } } : {}),
    footer: { text: "Apple Deal Scanner · Vinted" },
    timestamp: new Date().toISOString(),
  };
}

// ---------- Mémoire anti-doublons ----------
// seen.json garde : les identifiants d'annonces déjà traitées + la « signature »
// des annonces envoyées, pour ne pas renvoyer une annonce republiée ou vue dans
const SEEN_FILE = "seen.json";
const seen = new Set<string>(existsSync(SEEN_FILE) ? JSON.parse(readFileSync(SEEN
const saveSeen = () => writeFileSync(SEEN_FILE, JSON.stringify([...seen].slice(-2
const rejected = new Set<string>();
const signature = (v: Verdict, price: number) => `sig:${v.title.replace(/[^a-z0-9
let sentTotal = 0;

async function handle(items: Listing[], host: string, silent: boolean): Promise<n
  let fresh = 0;
  for (const l of items.reverse()) {
    if (seen.has(l.id) || rejected.has(l.id)) continue;
    fresh++;
    const c = check(l);
    if (typeof c === "string") {
      rejected.add(l.id);
      if (rejected.size > 40000) rejected.clear();
```

```
      if (!silent) console.log(`🚫 ${c} | ${l.price}€ | ${l.title.slice(0, 70)}`)
      continue;
    }
    const ref = marketPrice(c.p); // prix marché AVANT d'ajouter cette annonce
    addSample(l.id, c.p, l.price);
    seen.add(l.id); // marquée tout de suite : jamais envoyée deux fois
    if (silent) continue;

    const good = ref ? l.price <= ref.value * DEAL_RATIO : l.price <= (config.unk
    if (!good) {
      console.log(`💤 pas assez bas | ${l.price}€${ref ? ` (marché ~${Math.round(
      continue;
    }
    const sig = signature(c, l.price);
    if (seen.has(sig)) {
      console.log(`♻️ doublon ignoré | ${l.price}€ | ${l.title.slice(0, 60)}`);
      continue;
    }
    seen.add(sig);
    try {
      await sendToDiscord(dealEmbed(l, c, ref, host));
      sentTotal++;
      console.log(`➡️ ${countryOf(`https://${host}`)} | ${l.price}€${ref ? ` (mar
    } catch (e) {
      console.error("Discord :", (e as Error).message);
    }
    await sleep(700);
  }
  return fresh;
}

// Une « tâche » = une recherche dans un pays. Sa 1re lecture est silencieuse :
// elle mémorise ce qui est déjà en ligne pour n'envoyer que les NOUVELLES annonc
interface Job { host: string; s: Stream }
const ready = new Set<string>();
async function runJob(j: Job) {
  const id = `${j.host}|${j.s.query}`;
  const silent = !ready.has(id);
  try {
    const fresh = await handle(await searchVinted(j.s.query, j.s.catalog, j.host,
    ready.add(id);
    // Beaucoup de nouveautés d'un coup : on lit aussi la page 2 pour ne rien rat
    if (!silent && fresh >= 80) {
      await sleep(1500);
      await handle(await searchVinted(j.s.query, j.s.catalog, j.host, 2), j.host,
    }
```

```
  } catch (e) {
    console.error(`[${j.s.name} ${countryOf(`https://${j.host}`)}] :`, (e as Erro
  }
}

// ---------- Test hors ligne : SELFTEST=1 ----------
if (process.env.SELFTEST) {
  for (const t of (process.env.SELFTEST_TITLES ?? "").split("|").filter(Boolean))
    const [title, price] = t.split("@");
    const c = check({ id: t, source: "vinted", title, price: Number(price ?? 100)
    console.log(typeof c === "string" ? `🚫 ${c}` : `✅ ${c.p.cat} | ${c.p.key} |
  }
  process.exit(0);
}

// ---------- Planning des lectures ----------
// À chaque tour (~8-10 s) :
//   1) France / iPhone (le plus gros volume, jamais sauté)
//   2) iPhone dans un autre pays d'Europe, à tour de rôle
//   3) en alternance : iPad / MacBook en France, puis iPad / MacBook dans un aut
const main = config.streams[0];
const mainSide = config.streams.slice(1).map((s): Job => ({ host: MAIN, s }));
const abroadMain = OTHERS.map((host): Job => ({ host, s: main }));
const abroadSide = OTHERS.flatMap((host) => config.streams.slice(1).map((s): Job
const sideRotation: Job[] = [];
if (abroadSide.length) abroadSide.forEach((j, i) => { if (mainSide.length) sideRo
else sideRotation.push(...mainSide);

console.log("💎 Apple Deal Scanner démarré");
console.log(`📚 Prix marché déjà appris : ${Object.keys(market).filter((k) => !k.
console.log(`🌍 Pays : ${[MAIN, ...OTHERS].map((h) => countryOf(`https://${h}`)).

// Apprentissage au démarrage : 3 pages par recherche en France (silencieux)
console.log("⏳ Apprentissage des prix du marché…");
for (const s of config.streams) {
  for (let page = 1; page <= 3; page++) {
    try {
      await handle(await searchVinted(s.query, s.catalog, MAIN, page), MAIN, true
    } catch (e) {
      console.error(`[${s.name}] :`, (e as Error).message);
    }
    await sleep(3000 + Math.floor(Math.random() * 1500));
  }
  ready.add(`${MAIN}|${s.query}`);
}
saveSeen();
```

```
saveMarket();
const learned = Object.keys(market).filter((k) => !k.startsWith("famille:")).leng
console.log(`✅ Prêt : ${learned} modèles appris. Chaque nouvelle bonne affaire a

try {
  await sendToDiscord({
    author: { name: "✅ EN LIGNE" },
    title: "💎 Apple Deal Scanner",
    color: 0x5865f2,
    description:
      `📱 iPhone · 📲 iPad · 💻 MacBook — toutes générations, tous états\n` +
      `📊 ${learned} modèles avec un prix marché appris (il s'améliore en continu
      `✅ Envoi si prix ≤ ${Math.round(DEAL_RATIO * 100)} % du prix marché · 🔥🔥
      `🌍 ${[MAIN, ...OTHERS].map((h) => FLAG[countryOf(`https://${h}`)] ?? count
      `♻️ Chaque annonce n'est envoyée qu'une seule fois`,
    timestamp: new Date().toISOString(),
  });
} catch (e) {
  console.error("Discord :", (e as Error).message);
}

let tick = 0;
let rotA = 0;
let rotB = 0;
let lastLog = Date.now();
while (true) {
  const t0 = Date.now();
  await runJob({ host: MAIN, s: main });
  if (abroadMain.length) {
    await sleep(2000 + Math.floor(Math.random() * 1000));
    await runJob(abroadMain[rotA++ % abroadMain.length]);
  }
  if (sideRotation.length) {
    await sleep(2000 + Math.floor(Math.random() * 1000));
    await runJob(sideRotation[rotB++ % sideRotation.length]);
  }
  if (++tick % 10 === 0) {
    saveSeen();
    saveMarket();
  }
  if (Date.now() - lastLog > 5 * 60_000) {
    console.log(`📊 ${sentTotal} bonnes affaires envoyées depuis le démarrage`);
    lastLog = Date.now();
  }
  const wait = EVERY * 1000 - (Date.now() - t0);
  await sleep(Math.max(wait, 1500) + Math.floor(Math.random() * 1500)); // petit
```

```
}
```